

EXP 1 :Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

CODE :

```
import random

GRID_SIZE = 5

START = (0, 0)

GOAL = (4, 4)

OBSTACLES = [(1, 1), (2, 2), (3, 1)]

ACTIONS = {
    0: (-1, 0), # Up
    1: (1, 0), # Down
    2: (0, -1), # Left
    3: (0, 1) # Right
}

state = START

print("Starting Grid World Navigation")

while state != GOAL:
    print("Current Position:", state)
    action = random.randint(0, 3)
    move = ACTIONS[action]
    new_row = state[0] + move[0]
    new_col = state[1] + move[1]
    if new_row < 0 or new_row >= GRID_SIZE or new_col < 0 or new_col >= GRID_SIZE:
        reward = -5
        print("Moved Outside Grid")
        print("Reward:", reward)
    elif (new_row, new_col) in OBSTACLES:
        reward = -10
```

```

        print("Hit Obstacle")

        print("Reward:", reward)

    else:

        state = (new_row, new_col)

    if state == GOAL:

        reward = 100

        print("Goal Reached")

        print("Reward:", reward)

        break

    else:

        reward = -1

        print("Moved to:", state)

        print("Reward:", reward)

print("Navigation Completed")

```

OUTPUT :

Starting Grid World Navigation	
Current Position: (0, 0)	
Moved Outside Grid	
Reward: -5	
Current Position: (0, 0)	
Moved Outside Grid	
Reward: -5	
Current Position: (0, 0)	
Moved to: (0, 1)	
Reward: -1	
Current Position: (0, 1)	
Moved Outside Grid	
Reward: -5	
Current Position: (0, 1)	
Hit Obstacle	
Reward: -10	
Current Position: (0, 1)	
Moved to: (0, 2)	
Reward: -1	
Current Position: (0, 2)	
Moved Outside Grid	
Reward: -5	
Current Position: (0, 2)	
Moved Outside Grid	
Reward: -5	
Current Position: (0, 2)	
Moved to: (0, 3)	
Reward: -1	
Current Position: (0, 3)	
Moved Outside Grid	
Reward: -5	
Current Position: (0, 3)	
Moved to: (1, 3)	
Reward: -1	
Current Position: (1, 3)	
Moved to: (2, 3)	
Reward: -1	
Current Position: (2, 3)	
Moved to: (3, 3)	
Reward: -1	
Current Position: (3, 3)	
Moved to: (3, 2)	
Reward: -1	
Current Position: (4, 1)	
Moved Outside Grid	
Reward: -5	
Current Position: (4, 1)	
Moved to: (4, 2)	
Reward: -1	
Current Position: (4, 2)	
Moved to: (3, 2)	
Reward: -1	
Current Position: (3, 2)	
Hit Obstacle	
Reward: -10	
Current Position: (3, 2)	
Moved to: (3, 3)	
Reward: -1	
Current Position: (3, 3)	
Moved to: (4, 3)	
Reward: -1	
Current Position: (4, 3)	
Moved to: (4, 2)	
Reward: -1	
Current Position: (4, 2)	
Moved to: (4, 3)	
Reward: -1	
Current Position: (4, 3)	
Moved to: (3, 3)	
Reward: -1	
Current Position: (3, 3)	
Moved to: (4, 3)	
Reward: -1	
Current Position: (4, 3)	
Goal Reached	
Reward: 100	
Navigation Completed	

EXP 2 : Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

CODE :

```
import random

# True reward probabilities for 5 arms
true_rewards = [0.2, 0.4, 0.6, 0.8, 0.5]

trials = 500

epsilons = [0.1, 0.3, 0.5]

for epsilon in epsilons:

    estimates = [0] * len(true_rewards) # Estimated rewards
    counts = [0] * len(true_rewards)    # Number of times each arm is selected
    total_reward = 0
    explore = 0
    exploit = 0

    for _ in range(trials):

        # Exploration or Exploitation
        if random.random() < epsilon:

            arm = random.randint(0, len(true_rewards) - 1)
            explore += 1

        else:

            arm = estimates.index(max(estimates))
            exploit += 1

        # Get reward (1 = success, 0 = failure)
        reward = 1 if random.random() < true_rewards[arm] else 0

        total_reward += reward
        counts[arm] += 1

    # Update estimated reward
    estimates[arm] += (reward - estimates[arm]) / counts[arm]

    print("\n $\epsilon$  =", epsilon)
```

```
print("Total Reward    :", total_reward)
print("Average Reward  :", round(total_reward / trials, 3))
print("Explorations    :", explore)
print("Exploitations    :", exploit)
print("Best Arm Selected :", estimates.index(max(estimates)) + 1)
```

OUTPUT :

```
ε = 0.1
Total Reward      : 305
Average Reward    : 0.61
Explorations      : 42
Exploitations     : 458
Best Arm Selected : 4

ε = 0.3
Total Reward      : 355
Average Reward    : 0.71
Explorations      : 151
Exploitations     : 349
Best Arm Selected : 4

ε = 0.5
Total Reward      : 319
Average Reward    : 0.638
Explorations      : 226
Exploitations     : 274
Best Arm Selected : 4
```

EXP 3: Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

CODE :

```
import random

# Grid size
GRID_SIZE = 4

# Start and Goal
start = (0, 0)
goal = (3, 3)

# Energy available
energy = 10

# Actions
actions = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
}

# Transition probabilities
# 80% move correctly, 20% stay in same place
success_probability = 0.8

state = start
total_reward = 0

print("Autonomous Robot MDP Simulation")
print("Start State:", state)
print("Goal State:", goal)
print("Initial Energy:", energy)

while state != goal and energy > 0:

    action = random.choice(list(actions.keys()))
```

```

    print("\nChosen Action:", action)
if random.random() < success_probability:
    move = actions[action]
    new_row = state[0] + move[0]
    new_col = state[1] + move[1]
if 0 <= new_row < GRID_SIZE and 0 <= new_col < GRID_SIZE:
    state = (new_row, new_col)
energy -= 1
if state == goal:
    reward = 100
else:
    reward = -1
total_reward += reward
print("Current State:", state)
print("Reward:", reward)
print("Remaining Energy:", energy)
if state == goal:
    print("\nRobot reached the destination!")
else:
    print("\nRobot ran out of energy!")
print("Total Reward:", total_reward)

```

OUTPUT :

```

Autonomous Robot MDP Simulation
Start State: (0, 0)
Goal State: (3, 3)
Initial Energy: 10

Chosen Action: LEFT
Current State: (0, 0)
Reward: -1
Remaining Energy: 9

Chosen Action: LEFT
Current State: (0, 0)
Reward: -1
Remaining Energy: 8

Chosen Action: DOWN
Current State: (1, 0)
Reward: -1
Remaining Energy: 7

Chosen Action: UP
Current State: (0, 0)
Reward: -1
Remaining Energy: 6

Chosen Action: DOWN
Current State: (0, 0)
Reward: -1
Remaining Energy: 5

Chosen Action: LEFT
Current State: (0, 0)
Reward: -1
Remaining Energy: 4

Chosen Action: RIGHT
Current State: (0, 1)
Reward: -1
Remaining Energy: 3

Chosen Action: UP
Current State: (0, 1)
Reward: -1

Chosen Action: UP
Current State: (0, 1)
Reward: -1
Remaining Energy: 2

Chosen Action: DOWN
Current State: (1, 1)
Reward: -1
Remaining Energy: 1

Chosen Action: RIGHT
Current State: (1, 2)
Reward: -1
Remaining Energy: 0

Robot ran out of energy!
Total Reward: -10

```

CODE :

```
import numpy as np

grid_size = 4
gamma = 0.9
goal = (3, 3)
restricted = [(1, 1), (2, 2)]
values = np.zeros((grid_size, grid_size))
actions = [(-1,0),(1,0),(0,-1),(0,1)]
while True:
    delta = 0
    new_values = values.copy()
    for i in range(grid_size):
        for j in range(grid_size):
            if (i, j) == goal:
                continue
            if (i, j) in restricted:
                continue
            best = -1000
            for action in actions:
                ni = i + action[0]
                nj = j + action[1]
                if ni < 0 or ni >= grid_size or nj < 0 or nj >= grid_size:
                    ni = i
                    nj = j
                if (ni, nj) in restricted:
                    ni = i
                    nj = j
                reward = -1
```

```

        if (ni, nj) == goal:
            reward = 10
value = reward + gamma * values[ni][nj]
if value > best:
    best = value
new_values[i][j] = best
delta = max(delta, abs(values[i][j] - best))
values = new_values
if delta < 0.001:
    break
print("Optimal State Values")
print(values)
print("\nOptimal Policy")
policy = [["" for j in range(grid_size)] for i in range(grid_size)]
for i in range(grid_size):
    for j in range(grid_size):
        if (i, j) == goal:
            policy[i][j] = "G"
            continue
        if (i, j) in restricted:
            policy[i][j] = "X"
            continue
best = -1000
move = ""
directions = {
    "U":(-1,0),
    "D":(1,0),
    "L":(0,-1),
    "R":(0,1)
}
for name, action in directions.items():
ni = i + action[0]
    nj = j + action[1]

```



```
if ni < 0 or ni >= grid_size or nj < 0 or nj >= grid_size:
```

```
    ni = i
```

```
    nj = j
```

```
if (ni, nj) in restricted:
```

```
    ni = i
```

```
    nj = j
```

```
reward = -1
```

```
if (ni, nj) == goal:
```

```
    reward = 10
```

```
value = reward + gamma * values[ni][nj]
```

```
if value > best:
```

```
    best = value
```

```
    move = name
```

```
policy[i][j] = move
```

```
for row in policy:
```

```
    print(row)
```

OUTPUT :

```
=====
Optimal State Values
[[ 1.8098  3.122  4.58  6.2  ]
 [ 3.122  0.    6.2  8.    ]
 [ 4.58   6.2   0.   10.   ]
 [ 6.2    8.   10.   0.   ]]

Optimal Policy
['D', 'R', 'D', 'D']
['D', 'X', 'R', 'D']
['D', 'D', 'X', 'D']
['R', 'R', 'R', 'G']
```

EXP 5 : A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

CODE :

```
import random
battery = 30
deliveries = 0
total_reward = 0
actions = ["Deliver", "Wait"]
print("Warehouse Robot Simulation")
print("Battery:", battery, "minutes")
while battery > 0:
    action = random.choice(actions)
    if action == "Deliver":
        time_used = random.randint(3, 6)
        if battery >= time_used:
            battery -= time_used
            deliveries += 1
            total_reward += 10
        print("Action : Deliver")
        print("Time Used :", time_used)
        print("Battery Left :", battery)
        print("Reward : +10\n")
    else:
        print("Not enough battery for delivery")
        break
    else:
        battery -= 1
        total_reward -= 1
    print("Action : Wait")
    print("Battery Left :", battery)
    print("Reward : -1\n")
```

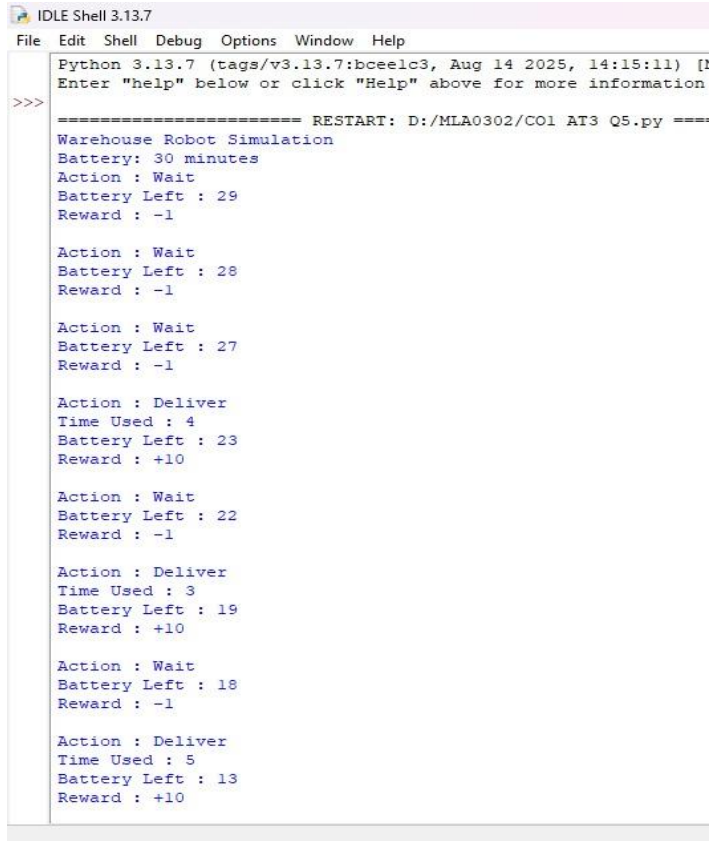
```
print("Simulation Completed")

print("Total Deliveries :", deliveries)

print("Total Reward :", total_reward)

print("Battery Remaining :", battery)
```

OUTPUT :



```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [1
Enter "help" below or click "Help" above for more information

>>>
===== RESTART: D:/MLA0302/CO1 AT3 Q5.py =====
Warehouse Robot Simulation
Battery: 30 minutes
Action : Wait
Battery Left : 29
Reward : -1

Action : Wait
Battery Left : 28
Reward : -1

Action : Wait
Battery Left : 27
Reward : -1

Action : Deliver
Time Used : 4
Battery Left : 23
Reward : +10

Action : Wait
Battery Left : 22
Reward : -1

Action : Deliver
Time Used : 3
Battery Left : 19
Reward : +10

Action : Wait
Battery Left : 18
Reward : -1

Action : Deliver
Time Used : 5
Battery Left : 13
Reward : +10
```